

Countermeasure graphs for software security risk assessment: An action research

Dejan Baca, Kai Petersen*

School of Computing, Blekinge Institute of Technology, Box 520, SE-372 25, Sweden

ARTICLE INFO

Article history:

Received 17 December 2012
Received in revised form 8 March 2013
Accepted 9 April 2013
Available online 18 April 2013

Keywords:

Software security
Risk analysis
Countermeasure graphs

ABSTRACT

Software security risk analysis is an important part of improving software quality. In previous research we proposed countermeasure graphs (CGs), an approach to conduct risk analysis, combining the ideas of different risk analysis approaches. The approach was designed for reuse and easy evolvability to support agile software development.

CGs have not been evaluated in industry practice in agile software development. In this research we evaluate the ability of CGs to support practitioners in identifying the most critical threats and countermeasures.

The research method used is participatory action research where CGs were evaluated in a series of risk analyses on four different telecom products.

With Peltier (used prior to the use of CGs at the company) the practitioners identified attacks with low to medium risk level. CGs allowed practitioners to identify more serious risks (in the first iteration 1 serious threat, 5 high risk threats, and 11 medium threats). The need for tool support was identified very early, tool support allowed the practitioners to play through scenarios of which countermeasures to implement, and supported reuse.

The results indicate that CGs support practitioners in identifying high risk security threats, work well in an agile software development context, and are cost-effective.

© 2013 Elsevier Inc. All rights reserved.

1. Introduction

Risk and threat analysis are an important part of early vulnerability prevention. Threat models (such as Peltier (2001) and Verdon and McGraw (2004)) and abuse cases (McDermott and Fox, 1999; McGraw, 2006) are different methods of identifying and preventing risks. Each of the approaches focuses on different angles, e.g. abuse cases (McGraw, 2006) and misuse cases (Sindre and Opdahl, 2005; Opdahl and Sindre, 2009) focus on the user perspective of the attacker, Peltier (2001) focuses on attacks and their probability and severity, and attack trees (Schneider, 1999) focus on goals and attacks. Overall, the challenge is to look at and incorporate all these perspectives. For example, when not considering a user group (also referred to as agents or agents), we might miss their goals, and with that potential threats. Furthermore, the previous models were general and did not consider the specific needs of agile development processes, these processes having small iterations and incomplete designs.

In response to the challenge Baca and Petersen (2010) proposed a combination of the user perspective (misuse and abuse cases) with the threat and risk perspective (Peltier and attack trees). The

approach incorporates attacker's goals, agents, attacks the agents can do, and countermeasures to prevent attacks. Furthermore, countermeasure graphs provide a mechanism to prioritize countermeasures, the priorities depending on the relative importance of attacker's goals, the likelihood of agents doing an attack, and the damage an attack can do, as well as the cost of preventing an attack. The approach was illustrated through an example of an online-game, but has not yet been evaluated in industry practice.

In this research we evaluated countermeasure graphs through participatory action research (cf. Kindon et al., 2007) in an industrial context where agile processes are used. In participatory action research researchers take an active part in introducing an intervention (in this case countermeasure graphs) into an organization. Based on observations and reflections during continuous use the intervention is progressively improved. Our choice of action research as a research method was motivated by the need of incorporating feedback into the approach and to evaluate improvements made progressively. Furthermore, the authors are employed at the company working with security related issues, which allowed them to easily participate actively in ongoing projects at the company. Overall, the action research makes the following contributions:

- Evaluation and improvement of countermeasure graphs from the user's perspective (in this case software developers who are not

* Corresponding author. Tel.: +46 455385877.

E-mail addresses: dba@bth.se (D. Baca), kai.petersen@bth.se (K. Petersen).

security experts using countermeasure graphs for risk assessment).

- Assessment of countermeasure graphs for software security in the context of agile software development, as no case studies evaluated the use of the previous approaches (Peltier, attack trees, and abuse cases) in that context.

Our proposition before evaluation was that the new approach (including tool support) is better suited to aid (agile) risk assessment as: (1) It eases the identification of countermeasures by allowing non-security experts to look at the security analysis from multiple perspectives (agent, goals, attacks, and countermeasures); (2) It stores traceability links between agents (also referred to as agents), goals, attacks, and countermeasures allowing to easily update and reuse risk analyses; (3) supports agile development by providing tool support and hence being able to automatically update and capture changes in the risk analysis immediately (e.g. when a countermeasure is implemented, the risk analysis situation changes). As a reference for an initial comparison to our approach we used the Peltier risk assessment approach (Peltier, 2001). Peltier was chosen as a reference for comparison as it was widely used at the studied company.

The remainder of the paper is structured as follows: Section 2 presents the related work of risk assessment approaches for software security. Section 3 illustrates how countermeasure graphs work. Thereafter, the research method (action research) is explained in Section 4. The results of the research are presented in Section 5, followed by summarizing the updates made to countermeasure graphs based on the evidence (Section 6), and a discussion of the results (Section 7). Section 8 concludes the paper.

2. Related work

The related work provides an overview of software engineering risk assessment in general, followed by risk assessment for software security. To complement the related work, definitions of terms are provided in Appendix A. Furthermore, we reviewed literature to identify checklists that could be used as aid in risk assessment.

2.1. Risk management in software engineering

Risks are associated with danger, threat, or loss. Furthermore, they are associated with uncertainty. In software development managing risk is divided into two main activities, namely risk assessment and risk control (cf. Boehm, 1991; Khan et al., 2012). The focus of this paper is on risk assessment for software security. In risk assessment risks are identified, analyzed, and prioritized (Boehm, 1991; Khan et al., 2012).

During the identification risk items are elicited and documented (Boehm, 1991). Different approaches have been used for that, such as documentation through use case diagrams and requirements (Sadiq et al., 2010), listing of risk factors and use of questionnaires (23), checklists (Boehm, 1991), and use of software metrics data (Chee et al., 1995).

Risk analysis determines the magnitude of the damage and the probability of a risk occurring (Boehm, 1991). To obtain the values, among others cost models (Boehm, 1991) and neural networks (Yong et al., 2006; Neumann, 2002).

During risk prioritization the risks are ordered according to their priority based on the outcome of the risk analysis. Different factors are taken into account when prioritizing risk, in particular risk priority, as well as cost-benefit analysis for risk reduction activities (Boehm, 1991). For prioritization with respect to the different variables different techniques are available (cf. Berander and Andrews, 2005), such as ranking (Berander and Andrews, 2005), Analytical

Hierarchy Process (Zahedi, 1986; Huang et al., 2004), or cumulative voting (Sawyer and MacRae, 1962; Baca and Petersen, 2010).

2.2. Software security risk elicitation and modeling

Peltier (2004) defines a risk assessment process for information security. The process consists of the steps: asset definition, threat determination, determine probability of occurrence of the threat, assess impact of the threat, recommended controls, and result documentation. By combining probability and impact into a matrix and placing the threats into that matrix a decision support is given to determine for which threat an action has to be taken (referred as controls, or in this paper referred to as countermeasures).

Schneier (1999) introduced attack trees, which is a notation to describe a system with respect to security. The root node of the tree describes what should be achieved (e.g. the goal is to access system) and the child nodes describe attacks of how that could be done (e.g. learn password). A child node can also have further child nodes of how to realize that attack (e.g. bribe administrator). Furthermore, a distinction is made between OR nodes (either choose one attack or the other) as well as AND nodes (do both attacks together). Each node (attack) can have a variety of attributes, such as its cost, equipment needed, and possibility to conduct the attack. From these one can then e.g. derive the most likely and cheapest low-risk attack, cheap high-risk attacks, etc. These then can guide which countermeasures to implement.

Misuse cases (Sindre and Opdahl, 2005) are defined based on UML use case definitions, a misuse case corresponds to a use case in UML, and an agent corresponding to a misuser. A misuse would correspond to an attack (in attack trees) or a threat (in Peltier's method), such as flood system, get privileges, etc. Alternatives of how to attack the system can be expressed in the form of misuse case variants. However, from an empirical standpoint a study using controlled experiments has shown that attack trees are more effective than misuse cases, even though people had similar opinions about them (Opdahl and Sindre, 2009). Complementary, abuse stories (also referred to as misuse story) have been proposed (Singhal, 2011; Roeser, 2011) in analogy to user stories.

Goal-oriented models have been introduced in the context of modeling software security. Lamsweerde extended the formal KAOS approach (Dardenne et al., 1993) for goal oriented requirements engineering to model security related goals (cf. van Lamsweerde (2004)). The goals are first formulated as high abstract goals, that are refined into subgoals through AND/OR refinement. That is, one can, for example, specify that all subgoals have to be achieved to realize a higher level goal. These are also referred to as system-to-go goals (cf. van Lamsweerde, 2004). Furthermore, Lamsweerde proposed to add anti-models to the goal model indicating obstacles that hinder goal achievement by violating the system-to-go goals. In order to express security goals specification patterns are suggested that are formalized through logic expressions. For the anti-goals one then should also answer who will benefit from that goal, and why an attacker would want to achieve the anti-goal. Once the obstacles are understood, a next step is to model obstacle resolution. Different strategies have been proposed (van Lamsweerde and Letier, 2000), such as goal substitution, goal weakening, anti-goal mitigation, etc. The approach has been illustrated using an example of a banking application.

Tropos Bresciani et al. (2004) is another goal-oriented approach that utilizes i^* (Yu, 1995) as a modeling approach to model the goals of agents and express dependencies between agents in achieving goals. Two types of goals are distinguished, namely soft-goals and hard-goals. Soft-goals are non-functional and qualitative, hence their achievement cannot be as easily determined. In i^* models security could be modeled in the form of soft-goals. However, no clear distinction is made between security and all other

requirements. In order to clarify the distinction Mouratidis and Giorgini (2007) propose extensions to Tropos to incorporate security in i^* . In particular, Mouratidis and Giorgini (2007) introduce security constraints, security dependencies, and security entities. To aid in the application of the approach Mouratidis illustrate the modeling approach through an industry example.

When prioritizing goals, attacks, and countermeasures approaches with trade-off are deemed useful, such as AHP (Berander and Andrews, 2005) or cumulative voting (Sawyer and MacRae, 1962). AHP does a pairwise comparison and the stakeholder has to decide by how much one item is more important than the other. Cumulative voting (also called 100\$ method) requires the user to assign dollars to items depending on their importance. If one item is twice as important as another item, it shall receive twice as many dollars. AHP only allows for a very small number of comparisons, hence it is not feasible in most cases (Berander and Andrews, 2005). Cumulative voting is also efficient when many items are to be prioritized (Berander and Andrews, 2005). Further details on the prioritization approach are provided in Section 3. In consequence, we proposed a new approach (countermeasure graphs) that (1) combines the ideas mentioned above, and (2) allows for relative prioritization (Baca and Petersen, 2010).

The above mentioned approaches are proposed as general risk assessment approaches, independently of any development methodology. Recently, Williams et al. (2010) proposed an approach that allows to assess the security risk associated with individual requirements. It is a simple to use approach based on the concepts of planning poker in agile teams on project level. For risk assessment Peltier's approach of combining risk and impact was used. The difference to countermeasure graphs is that they are not driven by individual requirements, but the system level perspective. However, a change of requirements might add new malicious agents, and hence change the relative risk of other identified threats/attacks.

2.3. Classifications and taxonomies for software security risk

As mentioned earlier risk assessment approaches rely on checklist and knowledge of known risks (cf. Boehm, 1991). For software security some checklists should be highlighted here that can be used to support the risk assessment process.

Tsipenyuk et al. (2005) introduced a taxonomy of typical software security errors that can be exploited by attackers. The so-called kingdoms represent lack of input validation and representation, API abuse, security features, time and state, errors, code quality, encapsulation, and environment. For each a number of vulnerabilities have been identified, e.g. buffer overflows could be a consequence of lacking input validation and representation.

Gilliam et al. (2003) introduced a list of ten common exploits to software systems, such as environmental variables, buffer overflows, script injection, race conditions, etc. Furthermore, they provide a checklist of 26 countermeasures that could be employed across the development lifecycle. These include system level countermeasures (e.g. avoid storing secret passwords), but also process related measures (e.g. the introduction of security metrics in the development process).

Besides having generic classifications domain specific ones have been proposed as well, given that security threats and countermeasures might depend on the domain.

Bierman and Cloete (2002) introduced a classification of threat classes and related countermeasures for the mobile application domain. Four main threat classes have been identified (integrity attack, availability refusal, confidentiality attack, and authentication risk). Within these a total of ten threat subclasses were presented, each having a list of countermeasures associated to it. For example, for theft of the mobile device temper resistant

hardware, trusted execution environment, itinerary recording, and sliding encryption were proposed, which are primarily relevant for the mobile domain.

Yeh and Chang (2007) structure countermeasures for information systems in three main countermeasures, namely IT-related, non-IT-related, and regulation and legality. IT related countermeasures include software, hardware, and data. Non-IT-related measures relate to physical facilities and personnel. Regulations relate to definition of security policies, compliance with legal requirements, and risk transference.

In the following section countermeasure graphs, which are the subject of evaluation in this paper, are further explained. In particular, we explain the pilot version with which we started the action research at the company.

3. Countermeasure graph

We combined the ideas of misuse cases (Sindre and Opdahl, 2005), attack trees (Schneider, 1999), and Peltier (2004) by considering agents and connecting them to goals, attacks, and countermeasures. In the following we describe the elements of countermeasure graphs, the relationships between the elements, and the steps to be conducted in using them. The method as described here was previously introduced in Baca and Petersen (2010).

3.1. Elements

The elements of countermeasure graphs are goals, agents (also referred to as agents in this paper), attacks, and countermeasures. The elements are defined as follows:

- **Goals:** Goals describe why anyone would attack the product and what their gain would be.
- **Agents:** Agents are users, roles and software that interact with the products. Preferably the entire range of possible users should be presented from end users to administrators, and outside software that interacts with the product.
- **Attacks:** After combining *Agents* with *Goals* we then look for *Attacks*. This step of the threat analysis is made easier because we have a clearer idea who would attack the system and why the attack would occur. We do not focus on whether the attack would work, only if it is a possible route for the *Agents* to take to achieve their desired *Goals*.
- **Countermeasures:** The final step is identifying what *Countermeasure* could be used to prevent an attack. A countermeasures can mitigate one or more attacks.

3.2. Relationships

Fig. 1 presents the relationships between the elements goals, agents, and attacks.

We consider the relationship between goals, agents, attacks, and countermeasures. In attack trees there exists a 1..* relationship between goals and attacks, i.e. one goal is related to several attacks while one attack is only related to one goal. However, in practice an attack could be executed by several agents, or an agent could pursue more than one goal. Hence, these relationships were extended to *..*.

Furthermore, we include priorities assigned to goals, agents, attacks, and countermeasures. The priorities are on ratio scale and are assigned by the person conducting the prioritization. The prioritization method is further explained when presenting the process of creating the countermeasure graph (see Section 3.3).

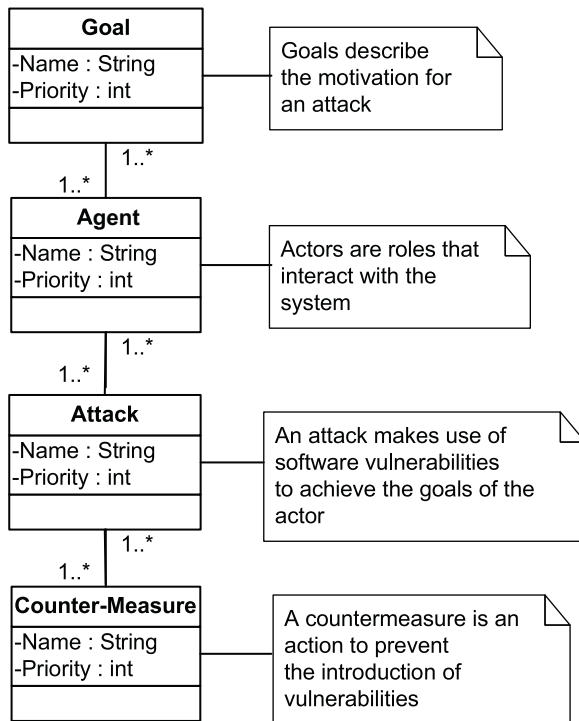


Fig. 1. Metamodel of the extension to attack trees.

We incorporate Peltier's ideas by including countermeasures (in his words called conduct), which are actions to avoid vulnerabilities to be introduced into the software product in the first place. Hence, from an implementation perspective it is important to prioritize the countermeasures, which is done by considering the priority of attacks, as well as the effort to realize the countermeasure. With the inclusion of countermeasures the protection of the end-product is supported.

An example of the countermeasure graph is shown in Fig. 3. The nodes of the graph show the goals, agents, attacks, and countermeasures. The edges connect:

- Goals to agents if the agent pursues the goal.
- Agents to attacks if the agent is likely to be able to execute the attack.
- Attacks to countermeasures if the countermeasure is able to prevent the attack.

3.3. Steps in using countermeasure graphs

Fig. 2 illustrates the steps to be conducted when applying countermeasure graphs. The figure also illustrates the approaches used in each step.

3.3.1. Identify goals, agents, attacks and countermeasures

The purpose of the countermeasure graph is to determine what security countermeasures would be useful to include in the product and at the same time identifying what countermeasures already exist. We determine what the greatest threats to the products are and how well we prevent them. To achieve this we need to discover what attacks can be executed and how they are stopped, with traditional threat analysis or attack trees this process is direct and tries to immediately identify attacks. In this approach, we ask developers to describe why one would attack the system (identifying the goals), who could attack the system (agents), how they could do it (attacks), and how that could be avoided (countermeasures).

The process of eliciting this information is similar to the one of eliciting requirements for a system. Commonly used elicitation techniques are interviews and workshops (Kotonya and Sommerville, 1998). In the case of security analysis, we used workshops as elicitation approach as identifying potential threats and countermeasures is a creative task, and the interaction of different developers with their different expertise might give creative impulses, e.g. the idea of one developer can trigger new ideas of another.

Having identified the goals, agents, attacks, and countermeasures we are interested in which countermeasures are most effective.

3.3.2. Prioritization of countermeasures

The countermeasure graph consists of sets of goals, agents, attacks, and countermeasures. In the following we present the prioritization questions asked to the developers, and also provide further explanation of the prioritization approach.

- **Goals:** *To what degree would the achievement of the goal damage the system from either a cost or stability perspective?* These are at first general goals that the different Agents might be interested in achieving. The Goals are connected to Agents that would be interested in achieving them. Voting on the Goals is focused on the damage that specific goal would sustain on the product. Depending on the goal the damage could be economical or system stability.
- **Agent:** *How large is the threat of the agent for the system?* With the Agents we determine how threatening the different roles are to the product. Votes are based on how "scared" the product should be of that Agent. As an example, a server product would consider end users more threatening than system administrators, while the opposite might be the case for privacy software.
- **Attack:** *How likely is the success of the attack for the agent it belongs to?* The attack voting is per Agent and determines how likely it is for that Agent to succeed with the attack.
- **Countermeasures:** *How efficient is the countermeasure in preventing an attack?* When voting on Countermeasures the focus is on their ability to prevent the attack compared to each other. In some cases two Countermeasures might be equally effective, but one of them might aid in preventing other attacks as well. The attack graphs would then put higher priority on that Countermeasure.

Each of the elements is prioritized according to the rules of hierarchical cumulative voting (HCV), originally introduced in Berander and Jönsson (2006) and experimentally evaluated in Berander and Svahnberg (2009). Cumulative voting has the benefit of (1) that it is simple to do, and (2) the distance of importance between two items is visible (cf. Berander and Andrews, 2005). The voting is done by providing a number of points (e.g. 100\$) and then distributing them between a set of items. In HCV the prioritization is done on each level of the hierarchy (in this case goals, agents, attacks, and countermeasures). Cumulative voting has been found to be a reliable approach for prioritization, and hence has been selected to support countermeasure graphs. It has also been used extensively for research purposes (cf. Rovegard et al., 2008; Kuzniarz and Angelis, 2011). Rovegard et al. (2008) investigated which issues arise in conducting change impact analysis, and applied HCV to prioritize them. Kuzniarz and Angelis (2011) investigated the relative importance of research problems on model consistency.

Fig. 4 shows the principle of HCV. The prioritization is done on all levels, in this case on level H0 no prioritization is necessary as there is only one node on that level (i.e. a single agent). On level H1 three nodes compete against each other for the points, i.e. nodes two (information modification), three (theft), and four (DoS). They represent the same element and hence are on the same level of

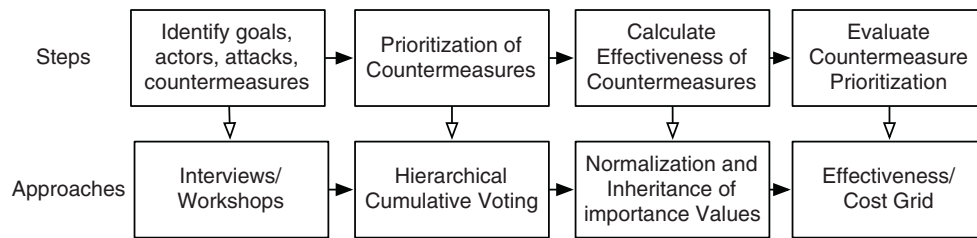


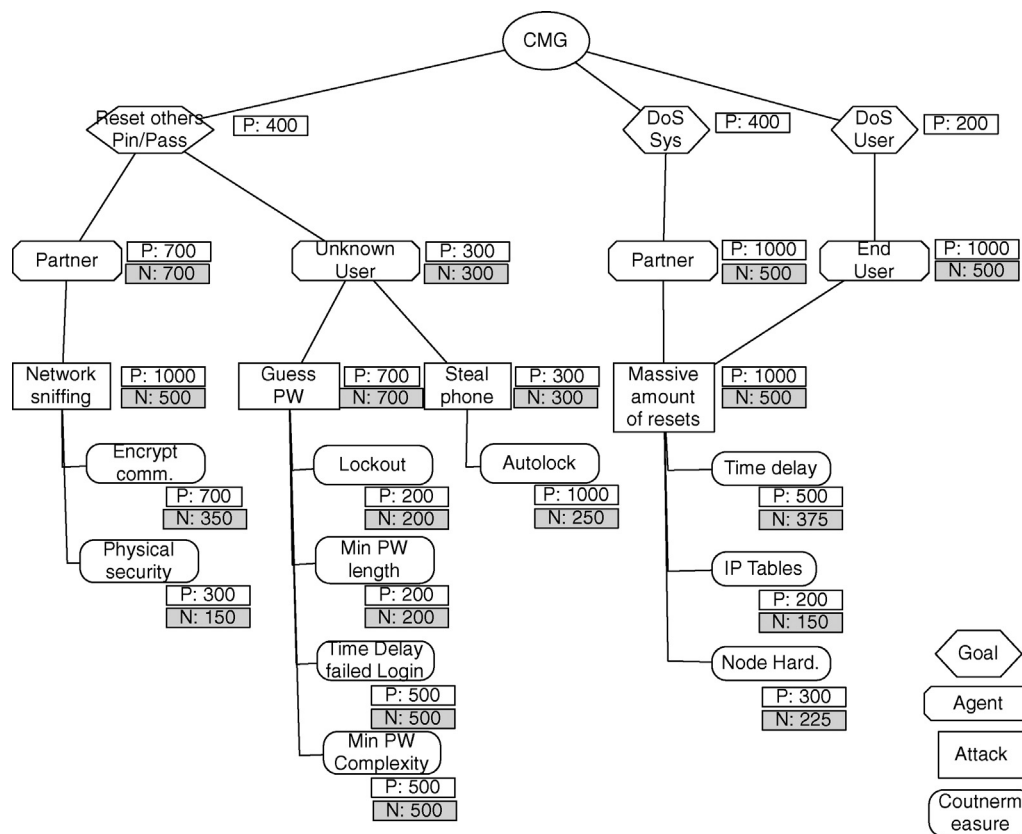
Fig. 2. Steps in using countermeasure graphs.

the hierarchy. As an example, H1 would represent the attacks and the prioritization would be done according to the question *How likely is the success of the attack for the agent it belongs to?*. The same approach is followed for each block (grey boxes) on the lower levels of the hierarchy, where only items within each block are prioritized against each other (e.g. in block H2.1 nodes 5 and 6 compete, in H2.2 nodes 7, 8, and 9, etc.). These represent the countermeasures, which are rated according to their efficiency in preventing an attack.

3.3.3. Calculate effectiveness of countermeasures

On the lowest level (H2) nodes within a block compete. Three blocks are identified, one for each node on the next highest level. Later one would, however, like to compare nodes in H2.2 to nodes in

H2.1. When comparing priority values between blocks (i.e. between different countermeasures in different blocks), then blocks with a high number of items are penalized as a given budget (e.g. 1000\$) is to be distributed among a higher number of items. Hence, Berander and Svahnberg (2009) conclude from their experiment that the use of a compensating factor is preferred. This is realized by first normalizing the priorities of item in each block. The normalization is calculated as: $X_{norm,y} = X_{prio} * n_y / m_{max}$, where $X_{norm,y}$ is the normalized priority of X in block y , X_{prio} is the priority assigned to X , n_y is the number of items in block y , and m_{max} is the maximum number of items in a block on the same level where X is located. Applied to the example in Fig. 4 the normalized value of node 5 is calculated as: $5_{norm} = 5_{prio} * 2/3$.



Example calculations to determine the effectiveness of countermeasures:

$$\text{Effectiveness(Autolock)} = 400 * 300 * 300 * 250$$

$$\text{Effectiveness(Time delay)} = (400 * 500 + 200 * 500) * 500 * 375$$

For "Time delay", two agents ("Partner" and "End User") are likely to do the attack, which increases the overall importance of the attack, and hence the effectiveness of related countermeasures. Hence, the values from "Partner" and "End User" are added.

Fig. 3. Countermeasure graph with priorities and effectiveness calculation.

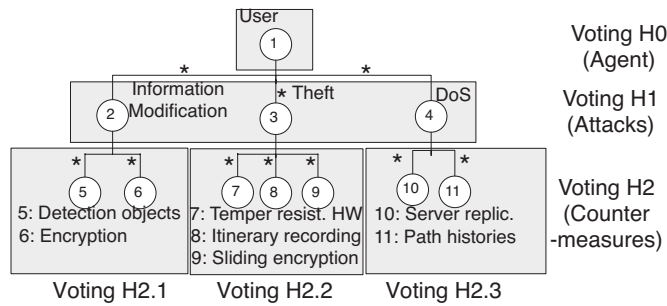


Fig. 4. Cumulative hierarchical voting.

When the priorities are normalized we compute the value of the node by multiplying the path of its parent nodes. For example, to know the value of node 11 we calculate the product of points assigned to node 11, 4, and 1.

Fig. 3 illustrates an example of a countermeasure graph including assigned priorities. At each node the hypothetical priorities given by practitioners are shown (white boxes besides the nodes), as well as the normalized values (grey boxes). Goals “Reset others Pin/Password”, “DoS Sys”, and “DoS User” are in one block and hence no normalization is necessary. However, on agent level the blocks related to “DoS Sys” and “DoS User” have only one agent each, and goal “Reset others Pin/Password” has two agents. Hence, values for the agents needed to be normalized. One important scenario should be highlighted, namely when an attack is related to two agents (such as “Massive amounts of resets”). Therefore, the importance of attack “Massive amounts of resets” increases, as the sum of the priorities of agent “Partner” and agent “End User” is calculated. Two calculation examples to determine the effectiveness of the countermeasures are provided at the bottom of Fig. 3.

3.3.4. Evaluation of result

Knowing the prioritization of the countermeasures with regard to effectiveness allows to combine them with costs in a matrix (see Fig. 5). The matrix shows different areas for the interaction of effectiveness and cost that can be used as support in deciding which countermeasures to focus on. The bottom-right area contains countermeasures that are effective and have low costs. Hence, these should be implemented first. The top-left area contains countermeasures that are ineffective and at the same time costly, hence they should be avoided. In the middle of the two areas the borderline-cases are shown, which could be implemented if there are enough resources available.

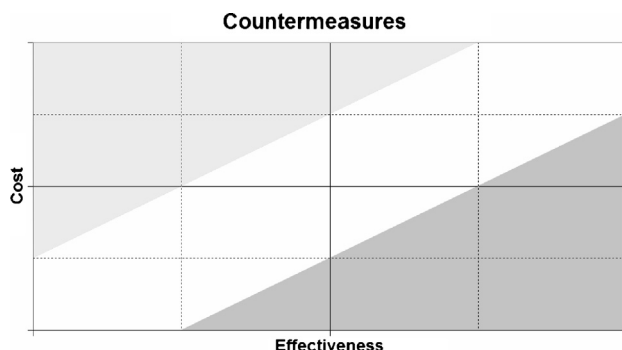


Fig. 5. Combination of effectiveness with cost.

4. Research method

The evaluation is conducted in an iterative way using action research. The steps of action research are planning of the intervention and the collection of the data to capture the intervention effect, the implementation of the actual intervention, and the collection of data and their interpretation (Somekh, 2006). As pointed out by Martella et al. (1999) much can be learned by continuously observing the effect of a change after inducing it. However, as the researcher is actively involved in the team work action research is an effort intensive approach from the researchers' point of view. In total we evaluated countermeasure graphs on four different products and six cycles to arrive at a mature and practically useful solution.

We followed the action research cycles of diagnosis (understanding the problem), action planning (plan an intervention to improve), action taking (implementing the intervention), and evaluation. The cycles of action planning, action taking, and evaluation are conducted throughout several cycles (cf. McKay and Marshall, 2001).

4.1. Research context

Describing the research context is of importance as it allows to understand the conditions under which the results were produced (Petersen and Wohlin, 2009).

4.1.1. Company

The company studied is a large telecom vendor (Ericsson LM). Ericsson offers solutions to support telecom operators in providing their services. The development unit studied is ISO certified. The company was working in a market-driven way, providing solutions to a large set of potential customers.

4.1.2. The agile process at the company

The process model used at the company was incremental and iterative in nature. Due to the introduction of incremental and agile development at the company the following company specific practices were introduced:

- **Small agile teams:** The first principle was to have small teams conducting short time-boxed projects. The duration of the project determined the number of requirements selected for a requirement package. Each project included all phases of development, from pre-study to testing. The result of one development project (DP) was an increment of the system. Projects can be run in parallel.
- **Prioritized requirements backlog:** The packaging of requirements for projects is driven by requirement priorities. Requirements with the highest priorities were selected and packaged to be implemented. Another criterion for the selection of requirements was that they fit well together and thus could be implemented in one coherent project.
- **Single branch with continuous integration:** If a project was integrated with the previous baseline of the system, a new baseline was created. This is referred to as the latest system version (LSV). Therefore, only one product exists at one point in time, helping to reduce the effort for product maintenance. The LSV could also be considered as a container where the increments developed by the projects (including software and documentation) were put together. On the project level, the goal was to focus on the development of the requirements while the LSV integrated packages of requirements to create the overall system. When the LSV phase was completed, a new version of the system is ready to be shipped.

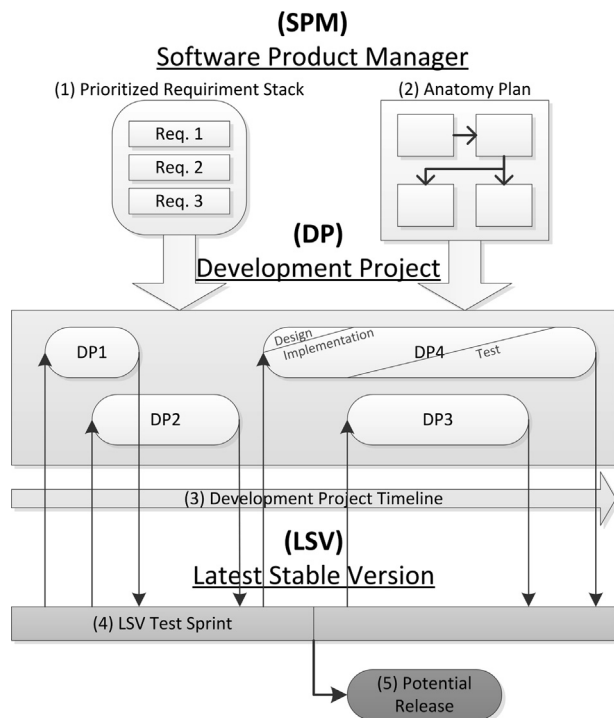


Fig. 6. An overview model of the agile processes used by the examined projects.

- *High level architecture plan (anatomy)*: The anatomy plan determined the content of each LSV and the point in time when a new LSV was supposed to be completed. It was based on the dependencies between parts of the system developed in DPs, thus influencing the time-line in which projects have to be executed.
- *Potential releases*: If every release was pushed onto the market, there would be too many releases used by customers at the same time that would need to be supported. In order to avoid this, not every LSV had to be released, but it had to be of sufficient quality for a potential customer release. The release project was responsible for making the product commercially available, performing any changes required to alter a development version into a release version.

In Fig. 6 an overview of the development process is provided. The requirements packages were created from high priority requirements stored in the repository. These requirements packages were implemented in projects resulting in a new increment of the product. Such a project was referred to as a Development Project and had a duration of approximately three weeks (time-boxed). Each DP contained design, implementation and testing of the requirements. It might happen that a DP was started in one sprint to later be released in another. When a project was finished developing the increment, the increment was integrated with the latest version of the system, referred to as last system version (LSV). From the LSV there could be different releases of the system. These were either potential releases or customer releases.

4.1.3. Security and risk assessment prior to introduction

Prior to the introduction of our approach the studied company conducted risk analysis as part of its development process. The risk analysis was modeled using Peltier risk analysis. The threats were rated with respect to likelihood of success and the severity of a successful attack. By multiplying severity and probability a risk level was produced.

After the threats were determined and ranked the most severe ones were analyzed to decide what action can be taken to

mitigate them. The conduct of this risk analysis was mandatory on all products at Ericsson and an essential part of the company's strategy to prevent vulnerabilities and have early vulnerability detection. It was recommended to conduct the risk analysis at the beginning of the development of a new software version.

Since the introduction of the risk analysis, projects in Ericsson moved from waterfall to incremental and agile development. The change of development processes led to shorter development cycles and new baselines of the product were more frequently created. These changes affected the risk analysis method as it relied on a fixed product plan that could be evaluated early in the process and then stayed stable until release.

4.1.4. Products

Four products have been investigated in a series of projects. In the following the products are shortly characterized. All products are developed using Java Programming Language.

- **Product A**: Had 5 development teams with 5–7 developers per team. One team specialized in verification and validation. The product had been in development for several years prior to the study. Risks analyses were only performed prior to major release versions.
- **Product B**: Had 3 development teams with 5–6 developers per team. One of the teams focused on verification and validation activities. The product was a spin-off from product A and used the same base framework.
- **Product C**: Had 3 development teams with 4–6 developers per team. All tasks were divided between the teams. Product development started during the study, meaning that no legacy risk analyses existed.
- **Product D**: Had 8 development teams with 6–8 developers per team. One team focused only on requirements and one only on verification and validation. The product had been in development for a year prior to the study.

Due to confidentiality reasons no further information regarding product development units or products can be provided.

4.2. Research question

The research goal was to *evaluate* countermeasure graphs in the context of practitioners conducting security threat assessment in agile software development *with respect* to their ability to identify security threats and countermeasures.

The following research question was in the center of analysis: *how should countermeasure graphs be modified to be useful in practice?* The answer to this question focused on what was working well with a version of the approach and what was not working well. Hence, the research allowed to provide explicit feedback on lessons learned while evolving the approach.

4.3. Action research design

Fig. 7 provides an overview of the four products under investigation, and the phases in which countermeasure graphs were evaluated. The vertical lines represent the timeline of investigation. The boxes represent projects run for the products, for which risk analysis was conducted.

In Cycle 1 (*Diagnosis*) Peltier was evaluated on products A and B, which were at that time analyzed as one product. The assessment was done by the developers and the result was reported in a document outlining the results of the different steps (asset definition, threat determination, probability rating, assessment of threat impact, recommended controls, and result documentation). In our analysis we focused on the outcome, i.e. the severity/probability

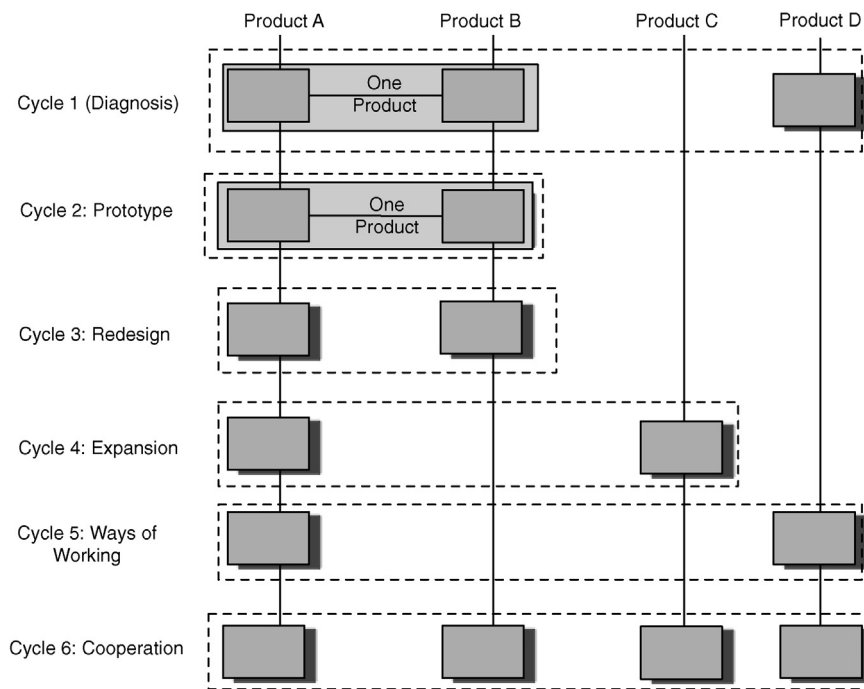


Fig. 7. Action research approach.

matrix to determine the threats identified. The severity rating determined the damage an attack does, while probability relates to the likelihood of a successful attack. Both were rated on a five-point scale. The values for severity and probability were then multiplied, resulting in the risk level (see Fig. 7)

In Cycle 2 (*Prototype*) the very initial version of countermeasure graphs (prototype) as presented in Section 3 was evaluated on product A. The first author was the moderator supporting the team in conducting the risk assessment using countermeasure graphs. The outcome (countermeasure graph and matrix) were used for analysis, as well as the observations and the feedback we received from the team. The team consisted of several developers that fulfilled different roles in the project. At a minimum one developer for each of these roles were present, namely requirement engineer, programmer, and tester. Often there was more than one developer per role and the risk analyses consisted of 5–6 developers including the author moderating risk analysis. After the risk analysis has been completed the author documented the outcomes of the meeting in a protocol capturing observations and results, as well as the feedback given by the practitioners. In Cycle 3 (*Redesign*) a new version of countermeasure graphs was evaluated in two projects, one related to Product A and one related to Product B. After the redesign we also saw a need for expansion, being evaluated in Products A and C (Cycle 4 (*Expansion*)). Finally, having a good understanding of how to design countermeasure graphs we used the experiences gained to define a way of working (Cycle 5 (*Ways of Working*)). That way of working was then evaluated in another project not related to Product A. During the risk analyses in the improvement phase the developers themselves decided when to conduct the analysis and which developers should be part of it. The author was no longer moderating the analysis, but was instead one of the developers providing expertise to the analysis. Since the author was still part of the analysis it was possible to observe problems and easily discuss improvements to the method and tool.

In Cycle 6 (*Cooperation*) we achieved a matured version of countermeasure graphs. After the improvement phase the tool developed to support countermeasure graphs was further extended to better support cooperation. The risk analyses continued after the

study had concluded and is part of the development process of the projects now.

5. Results

5.1. Cycle 1 (diagnosis)

Observations: Fig. 8 illustrates the results of the Peltier analysis. The figure shows that 20 threats with a risk level of low were found, and 5 threats with a risk level of medium. Related to Product A internal threats posed by the system administrator and hardware failures were identified. Only system threats are reported in the results, no product specific features were mentioned. The threats related to Product B were exclusively internal fraud by the system administrator and social engineering attacks from the outside. In order to increase the generalizability of the results, we investigated another risk analysis using Peltier that was conducted at the company (Product D). That product had 29 threats with the highest risk of 8. The threats were on system level and focused on the user accountability and shared account access. Overall, the results of the two analyses were similar in terms of Peltier's ability to detect threats when using developers that are not software security experts.

Evaluation: The highest risk factor that one could actually find is 25 (i.e. multiplying 5 (probability) * 5 (impact)). Overall, this showed that the Peltier approach only identified low risk/impact threats. However, it was likely that the systems had more significant security issues given their nature (complexity, exposed to networks, many different users). One potential reason for not finding more significant threats was that Peltier did not seem to steer the people in identifying the most critical ones. One potential reason for this was that when not combining the ideas from the different approaches, a more comprehensive and holistic picture of the threat situation is lost.

By observing the results of previous risk analyses and conducting a workshop with the developers, we identified potential reasons why the method (Peltier) did not identify higher level risks, and

Risk		Probability of successful attack					Risk Level	
Severity		1: Very low - Will almost never materialize.	2: Low - Not likely to materialize in near future.	3: Medium - Could materialize in near future, but frequent occurrence not expected.	4: High - Could materialize anytime, even frequent occurrence possible.	5: Very high - Frequent materialization expected soon after deployment.		
	1: Insignificant - Insignificant losses expected.	3 1	1 2	1 3	1 4	5	20 Low	Low – The threat could have severe impacts but it is highly unlikely that it will materialize, or the threat is expected to materialize frequently but the potential impacts are insignificant.
	2: Minor - Minor losses expected.	1 2	5 4	2 6	8	12	5 Medium	Medium – The threat could have severe impacts but it is not expected to materialize in the near future (at least not frequently), or the threat is expected to materialize frequently but without significant impacts on the operator's business.
	3: Significant - Significant losses expected.	3 3	2 6	9	12	14	0 High	High - The threat could have severe impacts but frequent occurrence is highly unlikely, or the threat is expected to materialize frequently but the expected losses are not disastrous.
	4: High - High losses expected.	5 4	1 8	12	16	20	0 Serious	Serious – The threat is expected to materialize frequently with serious consequences.
	5: Severe - Severe losses expected.	5	10	15	20	25		

Fig. 8. Peltier analysis (Cycle 1): Numbers in circles in the fields of the matrix show the risk level of the field (product of probability and severity rated on ordinal scale; values show the number of risks identified for each field in the matrix

focused only on specific types of threats (e.g. operating system level threats).

The results from previous risk analyses had often not yielded any risk with a higher rating than low and only in five occasions threats of medium risk level were identified. A root cause analysis revealed that the identified risks focused on OS level threats. The developers who performed the risk analysis acknowledged that they had problems to steer the risk analysis towards business logic threats instead of general system threats. The developers attributed this to lack of expertise and timing of the risk analysis.

While risk analyses should be performed early to identify threats before they have been implemented, this was seldom the case in the examined products. The main reason for late risk analyses was the uncertainty with regard to requirements. The teams responsible for risk analyses did not know if requirements would be implemented or how. As a consequence the risk analyses were performed shortly before a major release on already implemented features.

Another important reflection on Peltier is that from a measurement theory point of view the multiplication of distinct variables (ordinal) is not sound to calculate a risk level. The risk level also might increase in a non-linear way, which could not be expressed on the scale used.

5.2. Cycle 2 (prototype)

Action planning and taking: In the pilot two risk analyses were performed. For the first risk analysis we used post-it notes. The idea was to use the method in an as agile friendly way as possible. However, as the risk model grew it became time consuming to calculate the risk of a threat or the effectiveness of a countermeasure during the risk analysis session. Combined with the need to share and reuse the risk model the need for a tool became apparent. Therefore, a prototype was created and a second risk analysis was conducted on products A and B using the tool.

As these two risk analyses were the first instance with countermeasure graphs the focus was on adding the existing knowledge to them and to create a baseline for further analysis. As such the greatest focus was put on identifying all possible Agents that

interact with the product and any threatening Goals the Agents might have.

The risk analyses identified in Cycle 2 (Prototype) revealed 24 threats to the products (see Table 1). Some countermeasures were already implemented that mitigate threats identified by the users. For example, without the already implemented countermeasures attack A5 would have been in the serious risk category.

In comparison to the analyses conducted earlier, it was visible that threats of higher severity were identified. In particular, threat A22 was classified as serious, and A03, A06, A08, A04 as well as A16 as high. Furthermore, the number of medium threats increased from 5 to 11.

For the threats 37 countermeasures were identified, several of them prevent the same threats to different degrees (see Table 2). As such there were many more proposed countermeasures than attacks. Each countermeasure was also weighed on its effectiveness to prevent an attack and its cost of implementation. Therefore, it was possible to generate a list of the most important countermeasures, and also the degree of threat mitigation for a set of countermeasures.

The most significant threat to the system was related to faults and system errors (A22). To prevent this threat three countermeasures stood out. First, node hardening was recommended (c26). Hardening the node created a smaller attack surface and hence a safer product. The second countermeasure was to integrate a static code analysis tool into the build system and continuously classify and correct warnings (c24). Experiences of using static analysis in a security context have been reported in Baca et al. (2013). It was also required to correct all legacy warnings. The third countermeasure involved operative system side protection preventing buffer overflows and similar attacks on installed software (c7).

User security and passwords was the next big threat (A03), requiring basic settings like minimal password length and expired passwords (c34,c35).

The database was not sufficiently protected and was often subjected to user data (A16). It is therefore important to sanitize the user input before it was sent to the database. Two high value countermeasures protect the database: An easier database input

Table 1
Overview of threats and their severity.

Threat	Description	Peltier
A22	System attacks, buffer overflows, race conditions, ram reading and so on	Serious
A03	Login as another user	High
A06	Activate hidden processes that are not seen by system administrators	High
A08	Reading temp. & constant log files (file system)	High
A04	Creating backdoor via the end user portal	High
A16	SQL injection attack	High
A19	Bruteforce passwords	Medium
A01	Use specific log file for replay attacks, automated replay functionality	Medium
A20	Reuse old passwords	Medium
A24	Consume systems total number of session Id's	Medium
A13	Remove all or specific users	Medium
A15	Steal a session ID of a higher user	Medium
A12	Lock out other user accounts or Change other password	Medium
A21	Scan backups copies	Medium
A09	Retrieving information about the network topology	Medium
A18	Flooding the system or network with requests	Medium
A14	Force system back to default users and passwords (deleting all users)	Medium
A02	Login as user ad perform task add subscriber even if not supposed too	Low
A05	Perform tasks as another user/hacking access profile	Low
A07	Reading other operators database	Low
A10	Sniffing network traffic	Low
A11	Asking other users for their password	Low
A17	Fool a higher user to perform your tasks	Low
A23	Increase sequence number for cai(3g) commands	Low

cleaning solution that only sanitized data that is going into the database (c5); a more complete input validation that sanitized all data sent to the system (c3).

Furthermore, there was a lack of role protection within the OS level and several improvements have been suggested (e.g. c6).

The system supported locking users after a certain amount of failed logins. This opened a new attack where outsiders could lock any user they want (A12). Also the system had a limited number of sessions it allows, this again opens attacks so users can lock the system by using all sessions (A24). To prevent users from locking,

Table 2
Countermeasures: prio represents the relative priority of countermeasures in % of the total importance, PH represents the estimated effort in person hours

ID	Prio	PH	Threat Lvl	Description
c26	26.23	168	49%	Node hardening
c14	18.50	35	47%	Support expired password
c24	15.02	100	43%	Using static code analysis (automated analysis every build and constant classification of results)
c5	11.74	45	41%	Mandatory database input cleaning
c7	11.68	350	35%	SELinux or AppArmor
c34	8.81	21	34%	Minimum length passwords
c6	8.45	105	32%	Role based access control
c12	7.70	16	31%	Limiting tmp folder access and using private tmp folders
c35	6.95	44	30%	Minimum password complexity
c3	6.47	70	28%	Mandatory input validation
c23	6.01	300	24%	Use secure library (Secure C or Java, Type safe)
c36	5.96	45	22%	Per user max allowed session ids
c25	5.89	83	17%	Public/private key login (certificate logins)
c37	5.42	48	15%	Per IP max allowed session ids
c11	5.11	29	15%	Limiting log file read privileges
c1	3.79	43	14%	Lock session to specific lps
c30	3.75	45	13%	Restrict access to the replay log file
c29	2.60	65	12%	Restrict access to the replay function
c2	2.58	48	12%	User secure random generators
c33	2.53	45	11%	Time delay failed logins
c4	2.52	110		Input policy templates
c9	1.71	183		File system Intrusion detection system
c16	1.47	11		Limit access rights to Backup files
c8	0.92	750		Secure Kernel (grsec)
c31	0.87	68		Lock failed attempts per IP
c28	0.73	135		Verify replay logs checksum every read and write
c15	0.67	55		Recover to default states
c17	0.50	44		Encrypted backup copies
c13	0.41	238		One place control (or command) to remove Users from all possible sources, single sign on
c32	0.35	68		Lock failed attempts per browser session
c27	0.33	195		Network Intrusion detection system
c10	0.20	100		Encrypt Cai interface
c22	0.18	101		System that is forced to default setting automatically goes non-live or sounds every alarm possible
c20	0.13	80		Per user session restrictions
c18	0.11	60		Backup server that collects files system
c21	0.10	225		Prevent cause that would force system back to default settings or remove default setting functionality
c19	0.08	300		Network throttling/ traffic shaping

the “Time delay failed logins” countermeasure had the best effectiveness and man hour cost, it does not lock users, but instead puts a small (seconds) delay between every failed logon attempt (c33). A more costly alternative was to lock the failed attempt based on IP address (c1).

To prevent the session attack, we suggest setting a per IP address max allowed sessions and not a global max sessions (c36). Sessions should also be locked to a specific ip address (c37) to prevent users from stealing other logins, this includes the “Use Secure random” countermeasure. Another relevant threat we identified was the Log replay feature (A01) and the need to better protect the log file (c11).

The developers have identified user accountability as a major threat against the system. User accountability focuses on logging and verifying that operations are performed as they should. Harvesting passwords to steal other users accounts (e.g. through A03, A19, and A20) was seen as the goal that would create the greatest damage. Operative partners and system administrators were the Agents that presented the greatest threat to the system. As such the attacks and countermeasures focused on protecting users (e.g. c6), followed by DOS preventions (c33, c1, and c37).

The developers in the end had to make a decision which countermeasures to implement based on available resources. This was achieved by comparing the effectiveness of countermeasures to the cost (see Fig. 9). One countermeasure was clearly in the “implement” area, namely c26. Countermeasures in the decision area were e.g. c24, c6, etc.

Considering importance and cost, it was recommended that the following countermeasures were at least to be implemented:

- Node hardening (c26), protecting and minimizing threats to the OS as much as possible. Protect open ports, remove unnecessary services, update to latest versions and use encrypted protocols whenever possible. Protect the tmp (c12) folder and use secure tmp file handling. Use OS level security (c7 or c6).
- Implementing expired passwords (c14), minimum length passwords (c34). Implement password (C35) complexity on the entire system. Make sure secure random is used (c2) in any password or encryption generation.
- Use static code analysis (c24) as part of the development process and verify whether the warnings are corrected.
- Add input validation (c5 or c3) to the database input to clean all input from possible injection attacks.

Looking at the whole list of countermeasures, it would not have been cost efficient to implement countermeasures beyond c33 as the threat level was already significantly reduced, and only small improvements would be made for high cost.

Evaluation: Overall, it was visible that the approach allowed to identify high level threats to the system. Furthermore, a large number of countermeasures was identified and the developers were supported to make informed decisions of what countermeasures to implement.

The need for tool support was identified, hence we developed a tool and saw the following benefits of doing so: (1) The priorities were automatically calculated while practitioners enter the data; (2) the model can be easily reused and provided a knowledge base for risk analyses of similar systems.

5.3. Cycle 3 (redesign)

Action planning and taking: One change was made to the structure of the relationships of the model. The main improvement to the model was to decouple Agent and Goal. In the initial model a Goal was always connected to an Agent. The practitioners found it easier that way to add information to the model, and also found it more intuitive to connect goals directly to attacks to express what

the purpose of an attack directly. The change did not affect any weighting or require changes to previous risk analyses, as they did not have any attacks that required the specific case.

After the risk analysis in Cycle 2, the baseline for both products was present focusing on system legacy. Risk analysis in the Redesign cycle could hence focus on new implementations instead of legacy functionality for a new version of products A and B. With respect to the new features, each product was treated separately, as the analysis for their common platform and legacy was already done in the previous cycle.

For product A 5 new threats and countermeasures were added. However, these newly identified countermeasures and their related threats were already prevented by implementing countermeasures on system level, as identified in the risk analysis in Cycle 2 (Prototype). Hence, the identified attacks would have a high level of severance, but a low probability.

For product B 11 new threats (primarily related to authorization and authentication system) were identified and 13 new countermeasures were proposed. Of the 11 new threats three were considered as being of at least medium importance.

It was also the first time a suggested countermeasure created a new threat. In this case a suggested countermeasure against a Denial of Service attack meant that an IP address would be blocked, this made it possible for attackers to force the system to block ip addresses and thus creating a new Denial of Service attack.

The countermeasures from these risk analyses often solved the same problem. Because of this the developers could experiment within the tool and see what happened with the countermeasure prioritization list if they would implement one of the countermeasures. This aided the developers in deciding what countermeasure should be implemented and in what order. As an example, the third highest recommended countermeasure in the list would solve several of the other threats, but at the same time it would be costly do to, therefore lowering it in the list.

Evaluation: For product B the practitioners were able to identify additional threats/attacks of high importance on feature level, and found it easy to integrate/add them to the existing analysis. This showed that it was easy for them to reuse the already existing information in the model and to update it accordingly. The reuse of the model significantly reduced the effort of risk-analysis on feature level, and also assured to not implement countermeasures the system was already protected against.

Another observation was that the developers played through scenarios where they looked into what happens to the threat situation when certain countermeasures were implemented, especially if a set of countermeasures addressed multiple threats. This provided the opportunity to play through different scenarios for countermeasure implementation (e.g. with respect to order of implementation, what to implement for a fixed budget, and so forth), and underlined the need for a tool to support in recalculations of priorities.

5.4. Cycle 4 (expansion)

Action planning and taking: Based on the previous cycles we found that countermeasure graphs with tool support worked as intended, and produced high level threats and a variety of countermeasure alternatives to mitigate them. Hence, in the following cycles of the action research we focused on how to improve the tool and the feedback we received about working with a large model. By the end of this risk analysis iteration the different models consisted of 10+Goals, 10+Agents, 30+Attacks and 40+Countermeasures. The old data was still useful and needed in both the analysis and the calculation for prioritization, but at the same time it was cluttering the tool.

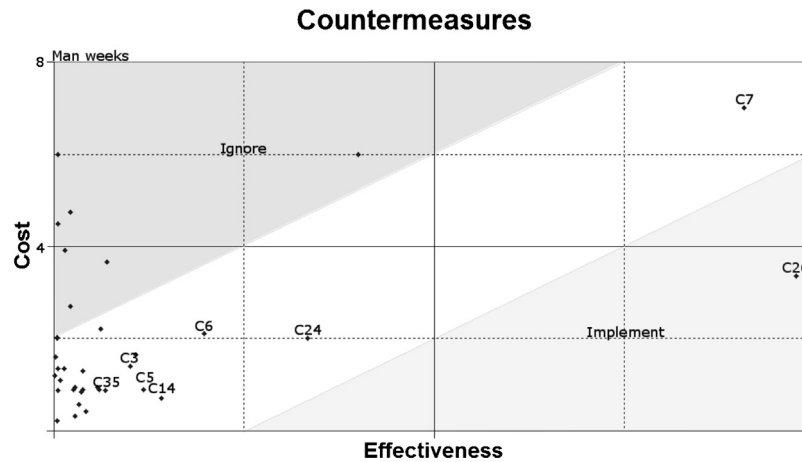


Fig. 9. Countermeasure effectiveness vs. cost.

Developers therefore recommend that attacks can be grouped together and that they can be marked as “mitigated” when countermeasures were implemented. By placing an attack as mitigated it was no longer used in the calculation for its countermeasures. Therefore, if an implemented countermeasure completely prevented an attack the risk analysis can now disregard the risk and related countermeasures as it was implemented. At the same time any countermeasure that was only protecting against that attack could be hidden from the graph and free up screen space. Attack groups were also implemented so that the developers could hide a group and all the Attacks and Countermeasures of those Attacks would hide and free up additional screen space.

In this iteration a third product has joined the study. The purpose of adding another product to the analysis was to increase the generalizability of the results. This new product did not share a platform or framework as products A and B do. However, it was still a telecommunication and client/server system using Java EE technology. As such it was possible for the developers to reuse most of the Goals, Agents and some of the Attacks of previous risk analyses, even if they were done for a different product. Therefore, the startup time for product C was considerably shorter than it was for products A and B. The first risk analysis with product C therefore immediately produced features based risks instead of just system level threats.

The threats from this iteration focused on the new multivendor support for product A and B, while product C was a completely new product and had several risks in its base requirements. All the newly identified risks were based on the product features instead of system level threats.

Evaluation: We found that the knowledge base for risks and related countermeasures was reusable when introducing a new product. Hence, this indicates that there is a high potential of reusability between similar products. Furthermore, the approach showed that additional high level risks could be identified.

5.5. Cycle 5 (ways of working)

Action planning and taking: In this iteration Product D joined the investigation of countermeasure graphs. Because the product had already conducted several major risk analyses (Peltier based) it first needed to move the results from the Peltier analysis to the tool. After that initial startup analysis it also produced feature based risks instead of the general system level threats.

With the inclusion of several projects it became necessary to consolidate and determine exactly what a Goal, Agent, Attack and Countermeasure was. It was therefore recommended by the

developers to include the description of the elements in the tool and to create a guide that the developers can follow when conducting the risk analysis.

Since product D was a relative new product and had high priority, it had a large amount of developers working with it. There was an interest in dividing the work of the analyses and not only focusing on a single analysis per sprint or release. Instead the identification of general Goals, Agents and Attacks was performed in the beginning of a new development cycle while the different development teams would take over the risk analysis and expand the model with new Attacks and Countermeasures based on the features they were implementing in that sprint. The tool and method supported this way of working and it was encouraged for the other products as well, but they did not see the need since they had so few teams.

Another recommendation by the developers was to use the data in the model to create a metric of the current security state of the product. By comparing what attacks have been prevented compared to the weighting of those that have not been prevented, it was possible to get a security metric based on the risks in the product.

Several new threats were identified and with the introduction of a new product, some severe system level threats were discovered as well.

Evaluation: After having conducted several iterations with the countermeasure graphs approach and tool, no new important changes were introduced to the approach anymore, and only minor additions to the functionality of the tool were made. This indicated that the solution was stabilizing, and functioning well for risk analysis.

5.6. Cycle 6 (cooperation)

Action planning and taking: At the end of the study each product continued to use countermeasure graphs as part of its development process. While the method itself has become stable, there have been some improvements in the way of working with the tool.

Since it was easy for product to share and reuse some parts of each other's risk analysis it was decided to have a general repository that the products could use to share and safely store their risks analyses. Since the tool uses a simple xml format to store the risk analyses it was possible to generate change logs and verify that risk analyses have been performed for each release. It also made it possible to tag the results the same way the source code was tagged, so that a risk analysis was associated with a certain source code release.

Because of the size and multi-site development of some of the products it was necessary to add network support to the tool. With the shared screen, result repository and the iterative way of working with the tool it was possible to perform risk analysis with teams that were not located in the same place.

The threats identified by the method have also focused on the new features instead rediscovering the same system threats that still are not important enough to correct.

Evaluation: In this cycle additional functionality was implemented to the tool, as many products were developed globally, hence supporting global software development. Recent research has emphasized the need for agile teams to work distributed, e.g. with shared screens (cf. Sharp et al., 2012).

In order to capture process adherence and making sure that risk analyses are conducted, the xml format supports in versioning and associating the risk analyses to different releases. This is important to base decisions on the correct (and always latest) version of the risk analysis, and also to support reuse (e.g. a risk analysis for a new product might be more comparable to the results of an early risk analysis of a mature product).

6. Updates to countermeasure graphs

In this section we summarize the changes made to the initial version of countermeasure graphs (see Section 3) based on the action research.

6.1. Elements

The set of elements as defined in Section 3.1 remained stable and did not require any redefinition. However, the elements received additional attributes.

- *Goals* have an extra monetary value that can be used to create an extra prioritization variable. Some developers wanted to divide the damage a goal could do to a system and the possible monetary losses.
- *Attacks* can be grouped so it is possible to focus on a certain type of attacks. If an attack was prevented it was made possible to deactivate it so that non-implemented countermeasures would not be prioritized if they protect against an already prevented attack.
- *Countermeasures* receive an additional variable to store cost, and also a boolean value that allows to capture whether the countermeasure has been implemented.

6.2. Relationships

The relations between goals, agents, and attacks have been changed. After the pilot we recognized that the relationship of Goal → Agent → Attack was not intuitive during the risk assessment process. The practitioners found it more intuitive to not being forced to identify an actor to express whether a goal can be achieved through a specified attack (in the current model a goal can not be attached to an attack if no actor has been decided at the same time). Hence, now it is easier to quickly add multiple actors and multiple goals in relation to an attack. It was also more natural to directly express that the goal is the purpose of the attack if they are directly connected.

Hence, we now connected agents directly to attacks, and goals directly to attacks. The nodes of the graph still show the goals, agents, attacks, and countermeasures. The edges connect:

- Goals to attacks if the purpose of the attack is the goal.
- Agents to attacks if the agent is likely to be able to execute the attack.

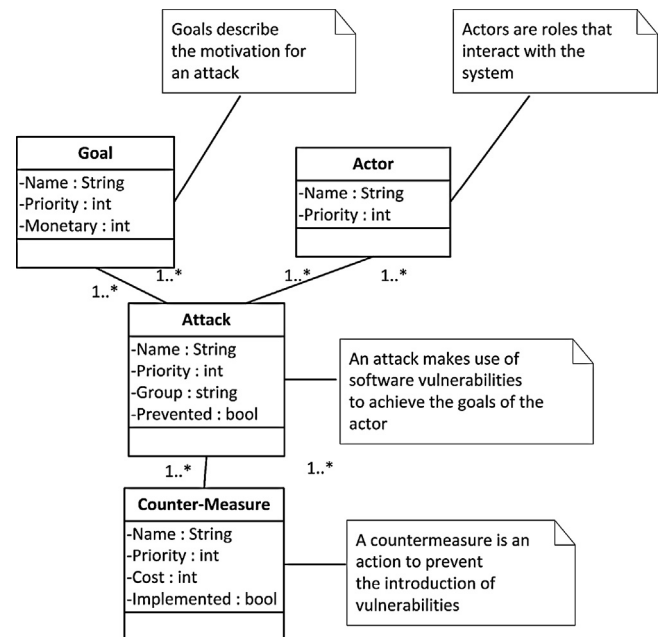


Fig. 10. Metamodel of the countermeasure graph.

- Attacks to countermeasures if the countermeasure is able to prevent the attack.

The meta-model for countermeasure graphs was updated accordingly. Fig. 10 shows the new meta-model, and Fig. 1 the old one. The attributes of the nodes have been updated according to the changes presented in Section 6.1.

6.3. Prioritization and calculation of effectiveness

There was no need to make changes to the principle of cumulative voting for calculating the severity of threats, and the effectiveness of countermeasures.

6.4. Evaluation of results

In the evaluation of the results, the matrix was working well in supporting developers to decide which countermeasures to implement, and the recommendation given was followed by them. We also recommend to use scenarios and check how the threat level changes for prioritizing different alternatives of countermeasures that address the same threats. Furthermore, playing with the scenarios supported the practitioners in deciding what to implement first.

6.5. Tool

In order to make countermeasure graphs work well the need for tool support was recognized very early in the action research. The finished tool is shown in Fig. 11. The values below the Goal and Agent names are their weighting compared to each other. The values in the arrows show the weighting for that particular edge.

The most important features added to the tool were:

- Storage of risk analysis results in XML-files to allow for reuse and change management.
- Update of attributes to allow indicating implemented countermeasures and mitigated attacks. This allowed to quickly play through different scenarios of implementing countermeasures.

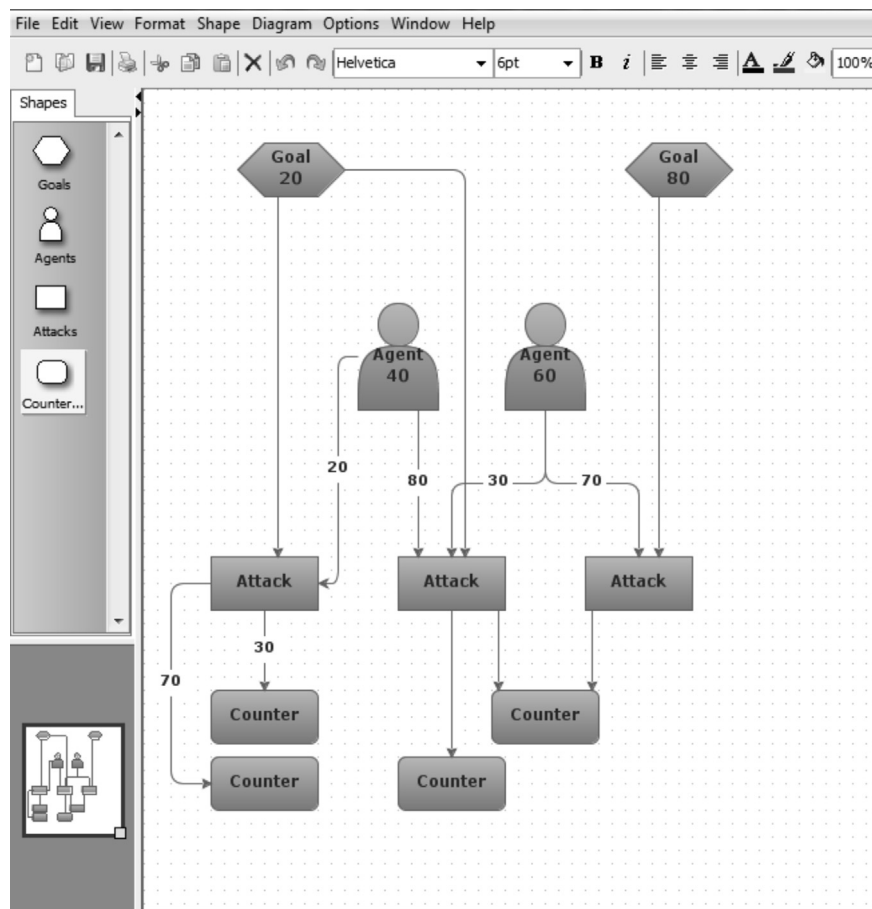


Fig. 11. Open source tool that implements the countermeasure graph.

- Implementation of attack groups that could be hidden to improve the layout and save screen-space when the countermeasure graph grows.
- Tool-tips defining the different elements to aid in using and learning the countermeasure graph approach.
- General repository for storing risk analyses across products to facilitate knowledge transfer between products.
- Network support to share the screen and being able to update the tool from different computers, due to that software development is often distributed.

7. Discussion

7.1. Ability to identify threats and countermeasures

The legacy risk analysis based on Peltier was conducted with developers having no specialization in software security. Using Peltier, no high level security risks were identified by them in the cases of products A, B, and C. The developers performing the risk analysis reported that they had problems identifying threats and that they did not always know how or what would be implemented at the time of the analysis. Some developers tried solving this by performing the risk analysis after implementation, but then they lost the entire benefit of detecting threats before implementing the design. Our proposition was that countermeasure graphs aid developers in identifying threats/attacks and countermeasures by combining the ideas of different risk assessment approaches.

The results of the action research provide indications for the proposition being true. In particular, comparing the Peltier analysis with the first risk analysis using countermeasure graphs, we found

that practitioners were identifying a higher number of threats with high risk level. In addition risk analysis on feature level, and for additional products, further high risk threats and related countermeasures were identified. An exception was product A in the redesign cycle (Section 5.3), where the newly identified threats on feature level were already resolved through implemented system level countermeasures. Identifying high level risks is particularly important as they are likely to harm the company's business more. With limited resources being available the risk level has to be taken into consideration when planning actions.

We observed that it is important to capture goals, agents, attacks, and countermeasures in a single risk model, as the identification of one of those elements aids practitioners in finding further elements. For example, given the awareness for a new agent added, practitioners came up with new ideas of what that agent could do. Furthermore, the identification of an attack spawned a new agent who could also realize the attack. Based on identifying that agent as relevant, new attacks related to the new agent could be identified. Hence, adding information to the risk model provides new ideas and helps extending the risk model further.

One limitation in the analysis was that we made this observations for developers without particular security experience and education. Which approach would be more effective for security experts can not be concluded based on this study. However, in practice we observed that organizations often have no particular role in relation to software security, even though this would be beneficial. Recently the studied company started to create security related roles (referred to as Security Master). In addition, universities start to offer Master and Bachelor programs focused on software security.

Potential validity threats for the observed observations are learning effect and confounding factors (see Section 7.5.4.).

7.2. Cost effectiveness

Risk analysis is a time consuming activity that requires people with good domain and product knowledge to participate. During the action research two types of reuse found to be beneficial. The first type refers to reusing the risk analysis for a specific product throughout increments of that product (as was the case in products A and B). This allowed the practitioners to focus on new risks while the product evolves (see Section 7.2). The second type refers to reuse between products. In the case of similar types of systems that were developed using the same platform (Java EE) we found that the practitioners were able to reuse the risk model to a large degree (as was the case for Product C in Cycle 4 (Expansion)), thus the practitioners were able to focus the risk analysis immediately on new features.

Given the reusability, the risk analyses are now stored in a repository so that the practitioners can access them easily for each new risk analysis conducted.

7.3. Reuse and evolvability of risk model supports agile development

Evidence on agile software development and how it is able to cope with security issues is scarce (Roeser, 2011). Roeser (2011) only identified two empirical studies on agile software security. Implementing agile security could be challenging. For example, Firesmith and Consulting (2003) recommends delaying architectural design decisions in relation to security due to that when committing to them too early, flexibility for change is lost. Given that agile software organizations have to deal with frequently changing requirements and releases, they are required to commit to design decisions on a frequent basis, and also be able to change the design when needed (cf. Petersen, 2010a). To what degree companies can change commitments might depend on the situation. For example, in the company case complex systems were studied, which had strict requirements on reliability, dependability, and security (e.g. guaranteed up-time is contractually fixed). Given the complexity of the systems and their high requirements on security, some high level architecture and design decisions (see anatomy plan in Fig. 6) need to be fixed early. At a later point it would be extremely expensive to restructure the system, which also affects the degree of agility and flexibility that could be achieved with such kind of systems.

In the studied case it was important to be able to continuously update the risk situation in an efficient way from a system point of view as the system evolved on a regular basis due to continuously incoming requirements. Therefore, CGs were designed with a focus on reuse and ease of maintaining and updating the risk model. As such CGs providing the developers with the possibility to focus the risk analysis on the impact of new features on system security and ignore mitigated legacy threats that were already in the model. The practitioners had the option to hide old countermeasures in the tool and add new information. This simplified the model. They were always able to make legacy visible if necessary to determine what happened in previous increments and iterations. This positively contributed to agile principles and practices. *Simplification of risk analysis*: By hiding old risk information not relevant for current or feature increments simplified the risk analysis model, and thus the ease of updating and reading it.

Planning and re-prioritization: By having the old threats in the calculations and reusing goals and agents it was easier for developers to focus and create threats for new features and increments. That is, by implementing a countermeasure in one increment

the risks are re-prioritized and new risks can be added for the subsequent increment. Such a situation occurred for products A and B, where in the first risk analysis system level threats were revealed and the subsequent risk analyses were identifying additional threats for new features added to the product's increments. Flexibility in planning is required in agile development (cf. Larman, 2003). Countermeasure graphs and the associated tool supported this situation, which was visible in Cycle 3 (Redesign) where the developers played through different scenarios of implementing countermeasures.

Reuse and development speed: Risk models can be reused by different products, which reduces lead-time and speed, which are considered essential measures in agile as well as lean software development (Petersen, 2010a,b; Larman, 2003). Furthermore, the time of an iteration is very limited. Thus, when wanting to conduct a risk analysis in an iteration, only a limited amount of time could be spent on it.

7.4. Improvements to countermeasure graphs

We used action research to continuously improve countermeasure graphs by examining and reflecting on each evaluation conducted. As the initial solution proposed might be faulty, it was iterated through several risk analyses. In retrospective we found that conducting several iterations of evaluation was important, as with each iteration new improvements were added. In the beginning the improvements were related to countermeasure graphs as such (see Sections 6.1 and 6.2), while later improvements primarily focused on supporting the risk analysis activity through the tool.

We saw two benefits of conducting an action research. First, by solving the problem in an iterative way the developers accepted the solution easily and integrated it into their way of working, and continued using it after the study. Second, by using several iterations a more stable and mature solution was presented for further technology transfer. Hence, we would highlight action research as a very useful way of conducting research projects together with industry.

Further improvements to countermeasure graphs can potentially improve them further.

One potential for improvement is to complement them with checklists. Examples of relevant checklists (generic ones as well as domain specific ones) have been presented in Section 2.3. Furthermore, risk analysis in this study was primarily conducted with more technical persons, while other perspectives (such as business perspective) could be useful to consider as well.

7.5. Validity threats

Four types of validity are commonly distinguished, namely construct validity, external validity, internal validity, and conclusion validity (cf. Wohlin et al., 2012).

7.5.1. Construct validity

Construct validity is concerned with obtaining the right measures for the concept being studied.

- *Reactive bias*: One threat in action research is that the presence of the researchers changes the natural behavior of the persons being observed. Hence, the researcher would affect the outcome of the study. This threat was reduced as the authors are both employed at the studied company and have been collaborating with the people in the company for several years. However, they were not part of their teams prior to the risk assessment, which reduces the risk that they accepting the new approaches as useful to please a colleague. Furthermore, the approach was continued

to be used without intensive involvement of the researchers in multiple products.

- **Correct data:** Another threat to validity is that the practitioners might understand countermeasure graphs incorrectly, and the wrong application would lead to incorrect data. To reduce that risk the practitioners were trained in the method, and the researchers were moderating the risk analysis to make sure the activities are followed correctly.

7.5.2. External validity

External validity is concerned with the ability to generalize the results.

- **Specific company:** The studies have been conducted in one company, and hence in a specific context, which is a threat to generalizability. We took two actions to reduce the threat. First, we describe the context carefully, allowing to judge under which circumstances the results were produced (as recommended in Petersen and Wohlin, 2009). Second, we studied several products to assure independence of the results from the context specific to a product.
- **Agile development context:** The development context studied was agile software development. The model allows updating and adjusting the priorities for countermeasures, and also allows hiding legacy analysis, which supports agile software development. The main difference to other development approaches is that they would not have so frequent updates and changes in the model; however, they would still require change management and would potentially have to deal with software maintenance after release. Hence, there is no indication that the model only ought to be useful in an agile development context.
- **Scale:** The products studied are of different scales when looking at the number of persons involved, ranging from more than 12 persons for product C to more than 48 for product D. The number of persons involved is a measure of scale in software development (Poile et al., 2009). The size of the teams in the range of 4–8 members, which is close to common team sizes for software development in general (cf. Pendharkar and Rodger, 2009), and agile software development in particular (cf. Salo and Abrahamsson, 2008).

7.5.3. Conclusion validity

Conclusion validity focuses on the ability to repeat and replicate the study, and whether the same result would be achieved conducting the study in the same setting.

- **Interpretation by the practitioners and the researcher:** The approach (countermeasure graph) relies on expert knowledge and judgement in order to determine threat levels and the effectiveness of countermeasures. This threat is always present as soon as human judgement is required. Three actions have been taken to reduce the threat. First, in each risk analysis several practitioners were involved, so that the result does not depend on individuals. Second, the authors reviewed the risk analysis, as they have expertise in security. Third, the analysis was conducted in several iterations and on several products, leading to similar results.

7.5.4. Internal validity

Internal validity is concerned with the ability to establish a cause and effect relationship between a treatment and a set of outcome variables. The threat here is that the researcher might be unaware of another cause that influences the outcome variable.

- **Confounding factors:** Whenever conducting industry research not all variables can be controlled (as is the case in controlled

laboratory experiments; Wohlin et al., 2012). One threat is, for example, that a factor other than the use of countermeasure graphs could have influenced the improved ability to find high-level threats and countermeasures. However, given that the team did not change, and hence the experience level was the same, this threat was partially reduced. A relevant confounding factor could be the researcher involvement in the research, and hence unconsciously influence practitioners in finding more severe threats. In order to reduce the threat, we made a conscious decision to stay in the moderator role and influence the subjects as little as possible, however, a potential unconscious influence cannot be ruled out completely, given that the authors are the developers of the approach.

- **Learning effect:** There is potential learning effect between assessments. In order to determine whether high-risk threats were identified as well without Peltier being used first, we investigated product C (triangulation). Overall, the research indicates that countermeasure graphs guide practitioners in also identifying high level risks. However, we cannot make claims regarding the size of the relative effectiveness of the approaches (countermeasure vs. Peltier) without a controlled experiment. There are indications that CGs helped security novices in finding more severe threats, as per their feedback and reflection on the results.

8. Conclusion

We have investigated how developers use software security risks analysis in an agile development process. The developers reported that they were not happy with the results given that mainly low level risks were identified. The short development cycles and uncertainty of the final design made it hard for them to conduct a useful risk analysis.

In an attempt to solve the problem we conducted action research the developers to improve a new risk analysis method (countermeasure graphs) and a tool supporting the method. Countermeasure graphs integrated the ideas of different risk analysis approaches (such as Peltier method and attack trees). We improved countermeasure graphs and the tool in an iterative way with the aid and feedback of the developers that conducted the risk analysis as part of their development process.

By first identifying roles that interact with the system and goals those roles might be interested to achieve it was easier for the developers to identify high risk attacks and related countermeasures. The method also created a prioritized list the developers could immediately use in their planning of short development cycles. If a new features was added to the product the developers could re-examine the existing Goals and Agents and see if any new attacks are possible with the new features. Furthermore, the use of the tool allowed the developers to play through different scenarios of implementing countermeasures, and they could observe how the threat level changes for each scenario.

After several projects had joined the study a matured countermeasure method and tool emerged, that was generalizable enough to work for all of the project and continued to be used after the study ended.

Overall, the results indicate that the solution supports practitioners in identifying high risk threats, is working well in an agile context, and is cost-effective as it supports reuse of previous risk analyses

Acknowledgements

We would like to thank all the participants in the study who participated in the action research. Furthermore, we thank the anonymous reviewers for valuable comments that helped in

improving the paper. This work has been supported by Ericsson AB and ELLIIT, the Strategic Area for ICT research, funded by the Swedish Government.

Appendix A. Definition of terms

All definitions are identical or close to how they have been presented in Matulevicius et al. (2008) where not stated otherwise. Matulevicius et al. (2008) has been chosen as it provides a consolidated conceptual model for security related terminology.

- **Agent:** An agent is an agent that can potentially cause harm assets (Matulevicius et al., 2008).
- **Goal:** Goals of attackers motivating why they would attack the system (own definition), in-line with the definition provided in van Lamsweerde (2004).
- **Attack:** An agent takes an action to exploit vulnerabilities of the system with harmful intend.
- **Asset:** The object of protection efforts. This may be variously defined as a system component, data, or even a complete system (McGraw, 2006).
- **Risk:** Risk is the combination of a threat with one or more vulnerabilities leading to a negative impact harming the assets (Matulevicius et al., 2008).
- **Threat:** A threat characterizes a potential attack or incident (Matulevicius et al., 2008).
- **Vulnerability:** A vulnerability describes a characteristic of an asset or group of assets that can constitute a weakness or a flaw in terms of security (Matulevicius et al., 2008).
- **Countermeasure:** Countermeasures are designed to improve security (Matulevicius et al., 2008) by threat mitigation.
- **Damage (also referred to as Impact):** Describes the potential negative consequence of a risk that may harm assets of a system or an organization, when a threat (or the cause of a risk) is accomplished (Matulevicius et al., 2008).

References

- Baca, D., Petersen, K., 2010]. Prioritizing countermeasures through the countermeasure method for software security (cm-sec). In: Product-Focused Software Process Improvement, 11th International Conference, PROFES 2010, Limerick, Ireland, June 21–23, 2010. Proceedings, pp. 176–190.
- Baca, D., Carlsson, B., Petersen, K., Lundberg, L., 2013]. Improving software security with static automated code analysis in an industry setting. *Softw. Pract. Exper.* 43 (3), 259–279.
- Berander, P., Andrews, A., 2005]. Requirements prioritization. In: Aurum, A., Wohlin, C. (Eds.), *Engineering and Managing Software Requirements*. Springer Verlag, Berlin, pp. 69–94.
- Berander, P., Jönsson, P., 2006]. Hierarchical cumulative voting (hcv) – prioritization of requirements in hierarchies. *International Journal of Software Engineering and Knowledge Engineering* 16 (6), 819–850.
- Berander, P., Svahnberg, M., 2009]. Evaluating two ways of calculating priorities in requirements hierarchies – an experiment on hierarchical cumulative voting. *Journal of Systems and Software* 82 (5), 836–850.
- Bierman, E., Cloete, E., 2002]. Classification of malicious host threats in mobile agent computing. In: Proceedings of the 2002 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists on Enablement Through Technology (SAICSIT 02), pp. 141–148.
- Boehm, B.W., 1991]. Software risk management: principles and practices. *IEEE Software* 8 (1), 32–41.
- Bresciani, P., Perini, A., Giorgini, P., Giunchiglia, F., Mylopoulos, J., 2004]. Tropos: an agent-oriented software development methodology. *Autonomous Agents and Multi-Agent Systems* 8 (3), 203–236.
- Chee, C.L., Vij, V., Ramamoorthy, C.V., 1995]. Using influence diagrams for software risk analysis. In: Proceedings of the Seventh International Conference on Tools with Artificial Intelligence (TAI 95), IEEE Computer Society, 28–.
- Dardenne, A., van Lamsweerde, A., Fickas, S., 1993]. Goal-directed requirements acquisition. *Science of Computer Programming* 20 (1–2), 3–50.
- Firesmith, D.G., Consulting, F., 2003]. Engineering security requirements. *Journal of Object Technology* 2, 53–68.
- Gilliam, D.P., Wolfe, T.L., Sherif, J.S., Bishop, M., 2003]. Software security checklist for the software life cycle. In: Proceedings of the 12th IEEE International Workshops on Enabling Technologies (WETICE 2003), pp. 243–248.
- Huang, S.-M., Chang, I.-C., Li, S.-H., Lin, M.-T., 2004]. Assessing risk in erp projects: identify and prioritize the factors. *Industrial management & data systems* 104 (8), 681–688.
- Khan, M.A., Khan, S., Sadiq, M., 2012]. Systematic review of software risk assessment and estimation models. *International Journal of Engineering and Advanced Technology (IJEAT)* 1 (4), 298–305.
- Kindon, S.L., Pain, R., Kesby, M., 2007]. *Participatory Action Research Approaches and Methods: Connecting People, Participation and Place*. Routledge, London.
- Kotonya, G., Sommerville, I., 1998]. *Requirements Engineering: Processes and Techniques*. John Wiley, Chichester.
- Kuzniarz, L., Angelis, L., 2011]. Empirical extension of a classification framework for addressing consistency in model based development. *Information & Software Technology* 53 (3), 214–229.
- Larman, C., 2003]. *Agile and Iterative Development: A Manager's Guide*. Pearson Education.
- Martella, R.C., Nelson, R., Marchand-Martella, N., 1999]. *Research methods: learning to become a critical research consumer*. Allyn and Bacon, Boston.
- Matulevicius, R., Mayer, N., Mouratidis, H., Dubois, E., Heymans, P., Genon, N., 2008]. Adapting secure tropos for security risk management in the early phases of information systems development. In: Proceedings of the 20th International Conference on Advanced Information Systems Engineering (CAISE 2008), Montpellier, France, June 16–20, 2008, pp. 541–555.
- McDermott, J.P., Fox, C., 1999]. Using abuse case models for security requirements analysis. In: 15th Annual Computer Security Applications Conference (ACSAC 1999), pp. 55–64.
- McGraw, G., 2006]. *Software Security: Building Security*. Addison-Wesley, Upper Saddle River, NJ.
- McKay, J., Marshall, P., 2001]. The dual imperatives of action research. *Information Technology & People* 14 (1), 46–59.
- Mouratidis, H., Giorgini, P., 2007]. Secure tropos: a security-oriented extension of the tropos methodology. *International Journal of Software Engineering and Knowledge Engineering* 17 (2), 285–309.
- Neumann, D.E., 2002]. An enhanced neural network technique for software risk analysis. *IEEE Transactions on Software Engineering* 28 (9), 904–912, doi:10.1109/TSE.2002.1033229.
- Opdahl, A.L., Sindre, G., 2009]. Experimental comparison of attack trees and misuse cases for security threat identification. *Information & Software Technology* 51 (5), 916–932.
- Peltier, T.R., 2001]. *Information security risk analysis*. Auerbach, Boca Raton, FL.
- Peltier, T.R., 2004]. Risk analysis and risk management. *Information Systems Security* 13 (4), 44–56.
- Pendharkar, P.C., Rodger, J.A., 2009]. The relationship between software development team size and software development cost. *Communications of the ACM* 52 (1), 141–144.
- Petersen, K., Wohlin, C., 2009]. Context in industrial software engineering research. In: Proceedings of the Third International Symposium on Empirical Software Engineering and Measurement (ESEM 2009), pp. 401–404.
- Petersen, K., 2010a]. Is lean agile and agile lean? A comparison between two software development paradigms. In: Dogru, Ali H., Bicer, Veli (Eds.), *Modern Software Engineering Concepts and Practices: Advanced Approaches*. IGI Global, Hershey, Pennsylvania, US, pp. 19–46.
- Petersen, K., 2010b]. An empirical study of lead-times in incremental and agile software development. In: Proceedings of the International Conference on Software Process, ICSP 2010, Paderborn, Germany, July 8–9, 2010, pp. 345–356.
- Poile, C., Begel, A., Nagappan, N., 2009]. Coordination in Large-scale Software Development: Helpful and Unhelpful Behaviors, Technical Report. Dept. of Computer Science, North Carolina State University, Raleigh, NC, September.
- F. Roeser, Can Software Security be Successfully Implemented in Agile Software Development? A Systematic Literature Review, Technical Report 11/01, Hochschule der Medien, Nobelstr. 10, 70569 Stuttgart, Germany (January 2011).
- Rovegard, P., Angelis, L., Wohlin, C., 2008]. An empirical study on views of importance of change impact analysis issues. *IEEE Transactions on Software Engineering* 34 (4), 516–530.
- Sadiq, M., Rahmani, M., Ahmad, M., Jung, S., 2010]. Software risk assessment and evaluation process (sraep) using model based approach. In: Proceedings of the International Conference on Networking and Information Technology (ICNIT 2010), pp. 171–177.
- Salo, O., Abrahamsson, P., 2008]. Agile methods in european embedded software development organisations: a survey on the actual use and usefulness of extreme programming and scrum. *IET Software* 2 (1), 58–64.
- Sawyer, J., MacRae, D., 1962]. Game theory and cumulative voting in illinois: 1902–1954. *American Political Science Review* 56, 936–946.
- Sawyer, J., MacRae, D., 1962]. Game theory and cumulative voting in illinois: 1902–1954. *American Political Science Review* 56, 936–946.
- Schneider, B., 1999]. Attack trees. *Dr. Dobbs Journal* 24 (12), 21–29.
- Sharp, H., Giuffrida, R., Melnik, G., 2012]. Information flow within a dispersed agile team: a distributed cognition perspective. In: Proceedings of the 13th International Conference on Agile Processes in Software Engineering and Extreme Programming (XP 2012), pp. 62–76.
- Sindre, G., Opdahl, A.L., 2005]. Eliciting security requirements with misuse cases. *Requirements Engineering* 10 (1), 34–44.
- Singhal, A., 2011]. Development of agile security framework using a hybrid technique for requirements elicitation, *Advances in Computing, Communication and Control* 125 (1), 178–188.
- Somekh, B., 2006]. *Action research: a methodology for change and development*. Open University Press.

- Tsipenyuk, K., Chess, B., McGraw, G., 2005]. Seven pernicious kingdoms: A taxonomy of software security errors. *IEEE Security & Privacy* 3 (6), 81–84.
- van Lamsweerde, A., Letier, E., 2000]. Handling obstacles in goal-oriented requirements engineering. *IEEE Transactions on Software Engineering* 26 (10), 978–1005.
- van Lamsweerde, A., 2004]. Elaborating security requirements by construction of intentional anti-models. In: *Proceedings of the 26th International Conference on Software Engineering (ICSE 2004)*, 23–28 May 2004, Edinburgh, United Kingdom, pp. 148–157.
- Verdon, D., McGraw, G., 2004]. Risk analysis in software design, *Security Privacy. IEEE* 2 (4), 79–84, doi:10.1109/MSP.2004.55.
- Williams, L., Meneely, A., Shipley, G., 2010]. Protection poker: The new software security “game”. *IEEE Security & Privacy* 8 (3), 14–20.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., 2012]. *Experimentation in Software Engineering*. Springer, Berlin.
- Yeh, Q.-J., Chang, A.J.-T., 2007]. Threats and countermeasures for information system security: a cross-industry study. *Information & Management* 44 (5), 480–491.
- Yong, H., Juhua, C., Zhenbang, R., Liu, M., Kang, X., 2006]. A neural networks approach for software risk analysis. In: *Proceedings of the Sixth IEEE International Conference on Data Mining (ICDMW 06)*, IEEE Computer Society, pp. 722–725.
- Yu, E., 1995]. *Modelling strategic relationships for process reengineering*, Dissertation, Department of Computer Science. University of Toronto.
- Zahedi, F., 1986]. The analytic hierarchy process a survey of the method and its applications. *Interfaces* 16 (4), 96–108.

Dejan Baca is a security engineer at Ericsson AB, Sweden. He received his PhD in computer science from Blekinge Institute of Technology in 2012, and his M.Sc. in computer science in 2004. His research interests include software security, software processes, game development, and agile software development.

Kai Petersen is an assistant professor in software engineering at Blekinge Institute of Technology, Sweden. He received his Ph.D. and Master of Science in Software Engineering (M.Sc.) from Blekinge Institute of Technology. His research interests include empirical software engineering, software process improvement, lean and agile development, software testing and software measurement. He has over 30 publications in peer-reviewed journals and conferences. He is the industry co-chair at REFSQ 2013, the 19th International Working Conference on Requirements Engineering: Foundations for Software Quality.